ICG:
Modelagem Geométrica

Profª. Tainá Isabela

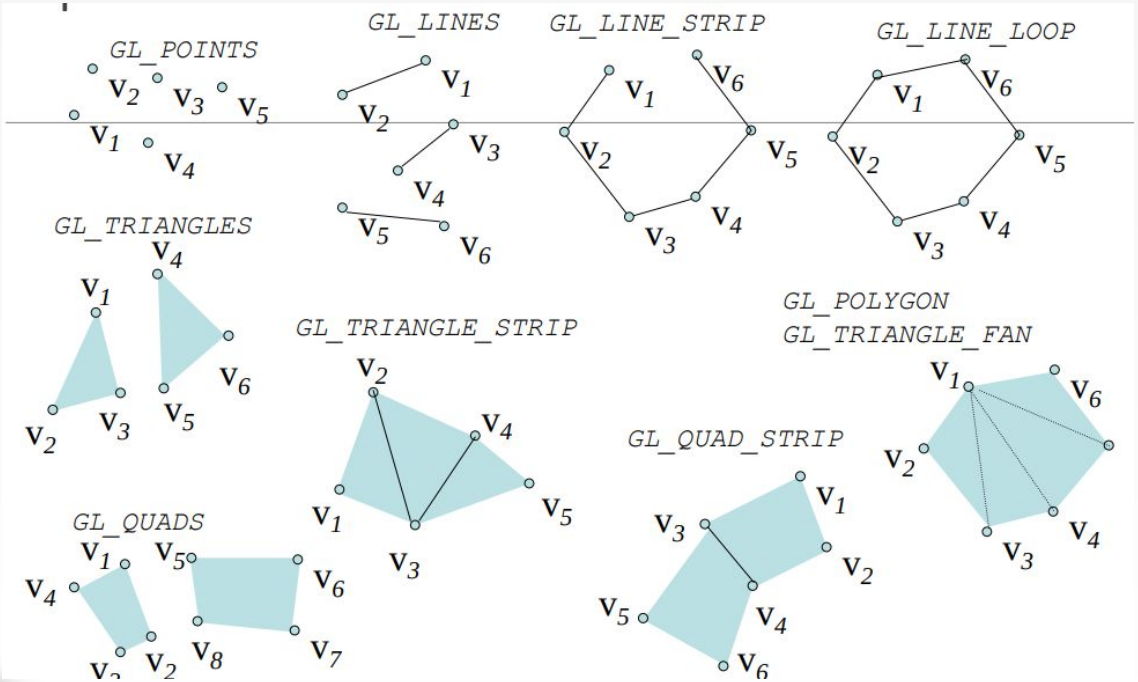
OpenGL – Primitivas de desenho

Entre glBegin() e glEnd() apenas alguns comandos podem ser usados. Ex.:

- `glMaterial;`
- `glNormal;`
- `glTexCoord;`

Uma vez emitido um vértice (glVertex), este é desenhado com as propriedades (cor, material, normal, coordenadas de textura etc) registradas nas variáveis de estado correspondentes.

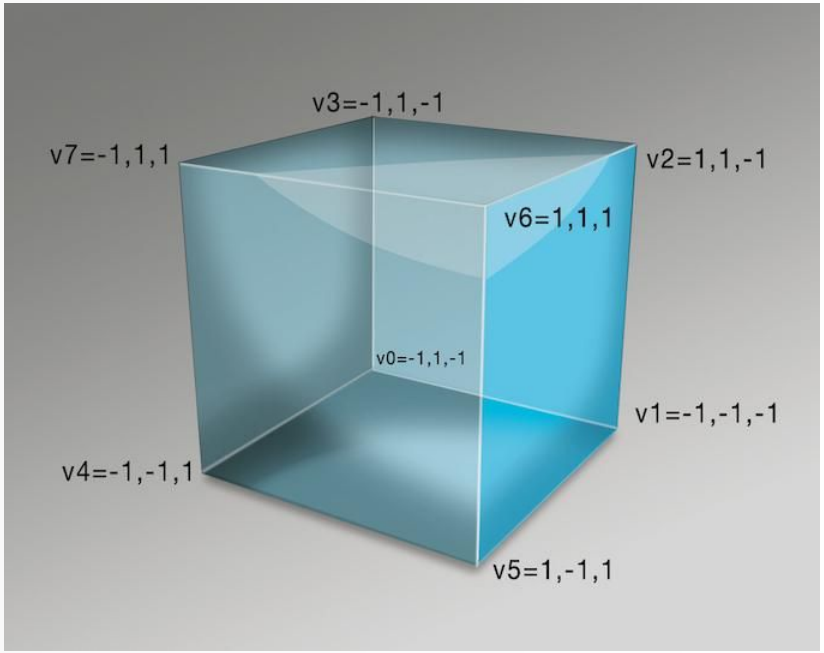
OpenGL – Primitivas de desenho



Como objetos 3D são representados?

Vértices (Vertices)

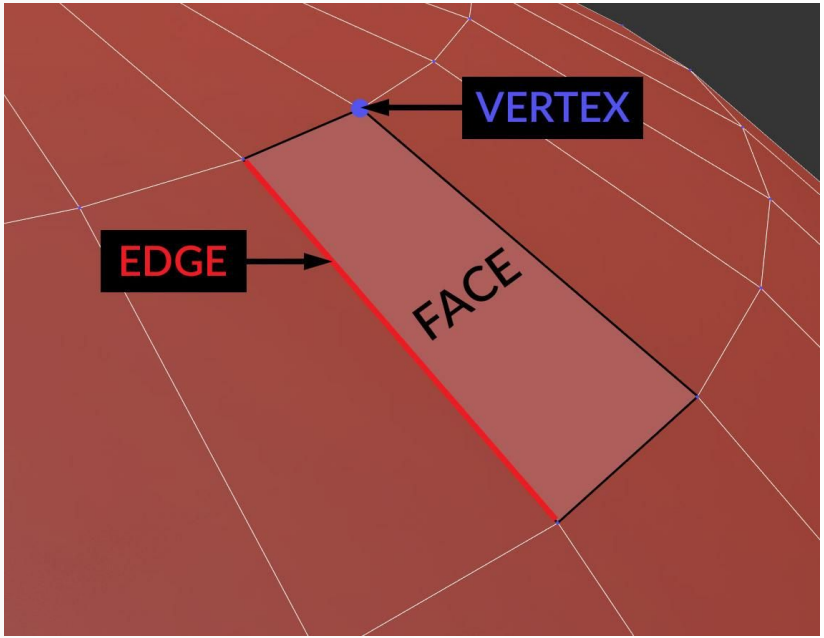
- São pontos no espaço 3D;
- Definidos por coordenadas: (x, y, z);



Como objetos 3D são representados?

Arestas (Edges)

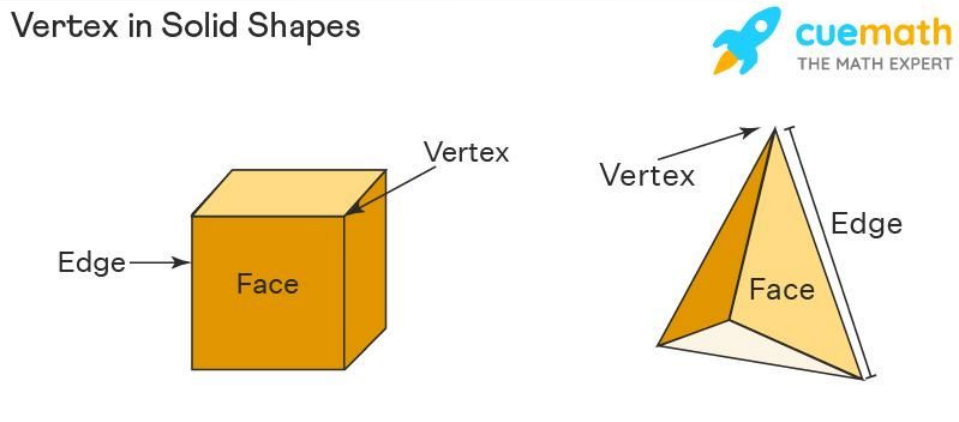
- São linhas que conectam vértices;
- Definem a estrutura do objeto;



Como objetos 3D são representados?

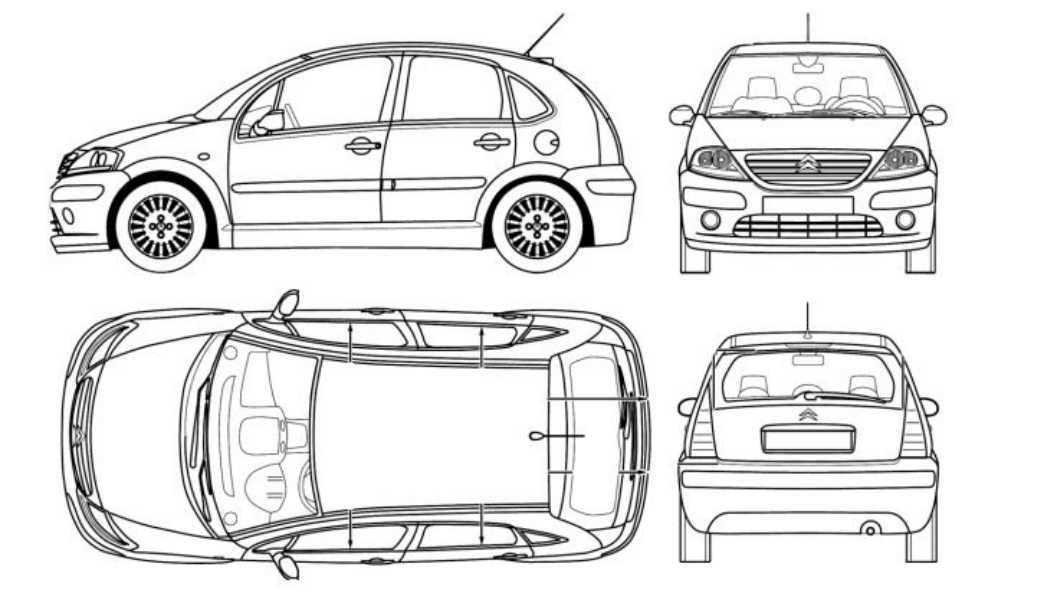
Faces (Faces)

- São superfícies formadas por vértices;
- Normalmente:
  - Triângulos (mais comum);
  - Quadrados (quads);



Modelando formas complexas

Podemos modelar uma esfera com uma equação, mas como ficaria uma equação para um carro?



Modelando formas complexas

Modelos complexos são criados com uso de:

- Polígonos, superfícies paramétricas e superfícies Implícitas;

Objetivo:

- Modelar qualquer coisa com certa precisão;
- Facilidade de implementação;
- Desempenho;

Malhas de polígonos

Qualquer forma pode ser modelada com o uso de polígonos

Polígonos com quantos lados?

- ◦ Triângulos são mais comuns.
- ◦ Quando > 3 lados são usados podem gerar polígonos não planares, côncavos ou complexos.

  Malhas são criadas com:

  - Vértices (pontos)
  - Aresta
  - Faces
  - Primitiva: GL_TRIANGLES

Malhas de polígonos

Manualmente:

- Escrever todos os polígonos;
- Escrever um código que gere os polígonos;
- Edição interativa;

Objetos reais:

- Scanners;
- Gera um nuvem de pontos que são convertidos para polígonos;

Estrutura de dados para malhas

Simples:

- Armazenar os 3 vértices de cada triângulo.
- Redundante: cada vértice será armazenado várias vezes.

Complexo:

- Lista de vértices , cada vértice é representado somente por suas informações geométricas (x,y,z) e possui um identificador (id).

- Lista de triângulos, cada triângulo é construído com três ids de vértices.

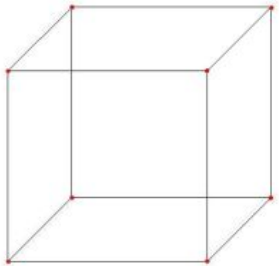
Tipos de malhas

- Wireframe (arame) → só arestas;
- Superfície (faces) → modelo sólido;
- Triangular mesh → padrão da indústria;

- Observações:
  - GPUs trabalham melhor com triângulos;
  - Tudo pode ser convertido em triângulos;

Arquivo OBJ para modelos de polígonos

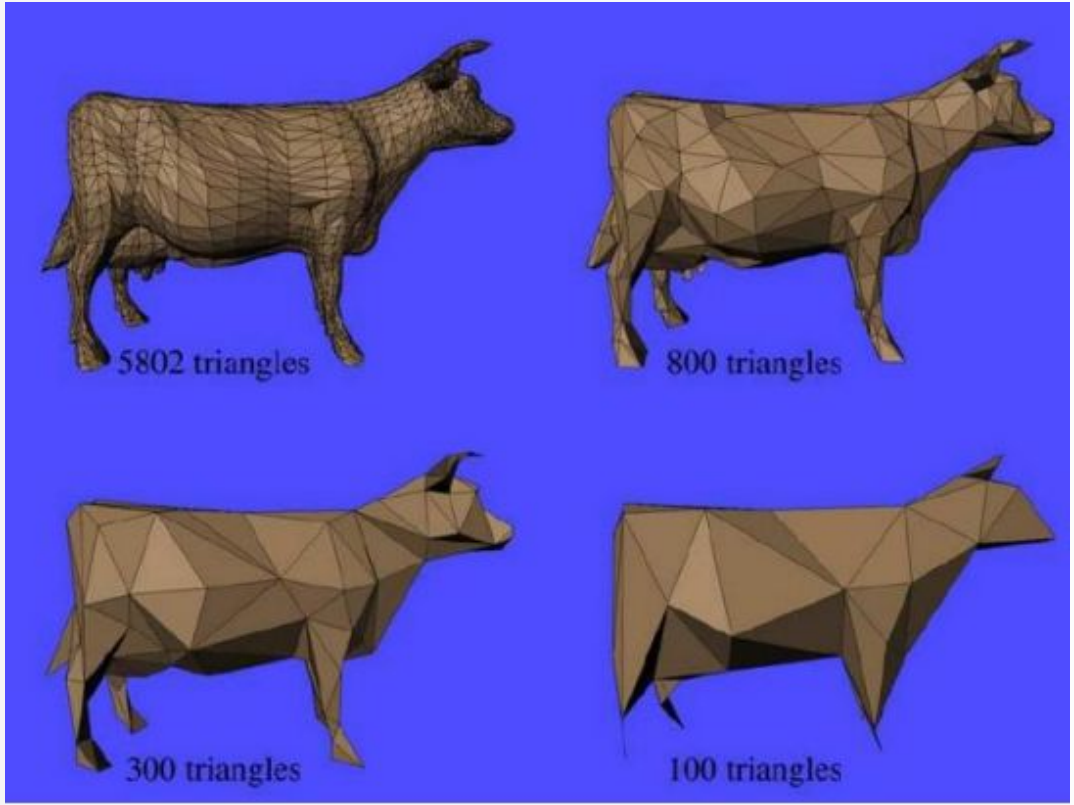
```
# OBJ cubo 2x2x2
o cubo
v -1.0 1.0 1.0          - vertex 1
v -1.0 -1.0 1.0         - vertex 2
v 1.0 -1.0 1.0          - vertex 3
v 1.0 1.0 1.0           -...
v -1.0 1.0 -1.0
v -1.0 -1.0 -1.0
v 1.0 -1.0 -1.0
v 1.0 1.0 -1.0
f 1 2 3 4
f 8 7 6 5
f 4 3 7 8
f 5 1 4 8
f 5 6 2 1
f 2 6 7 3
```



Sintaxe:

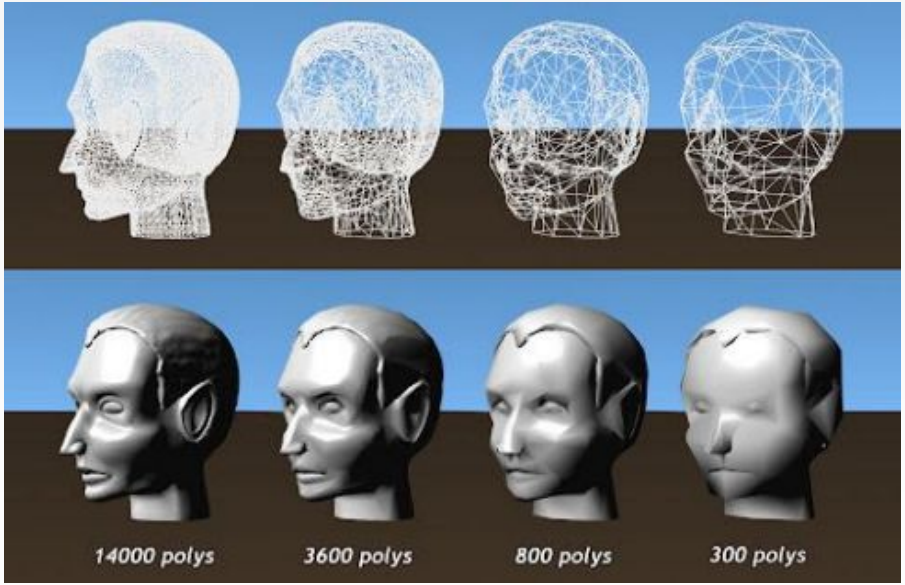
v x y z	- Um vértice em (x,y,z)
f v_1 v_2 ... v_n	- uma face com vértices v_1 v_2 ... v_n
# texto	- Comentário
o	- Objeto
vn x y z	- Vetor normal
vt x y z	- Coordenada de textura

Quantidade de polígonos influencia



Nível de Detalhe

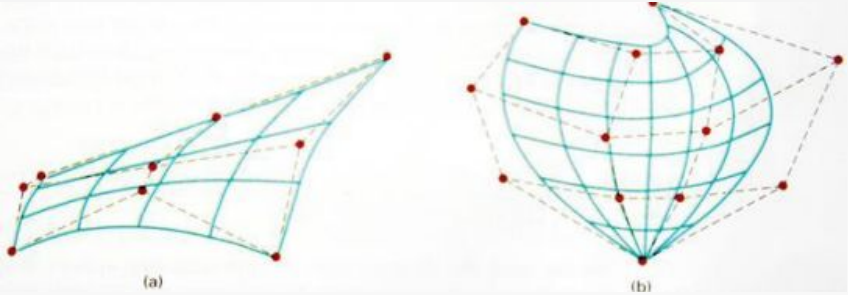
- Modelos diferentes para objetos próximos e distantes.
- Modelos diferentes para renderização e detecção de colisão.



Problemas com malhas de polígonos

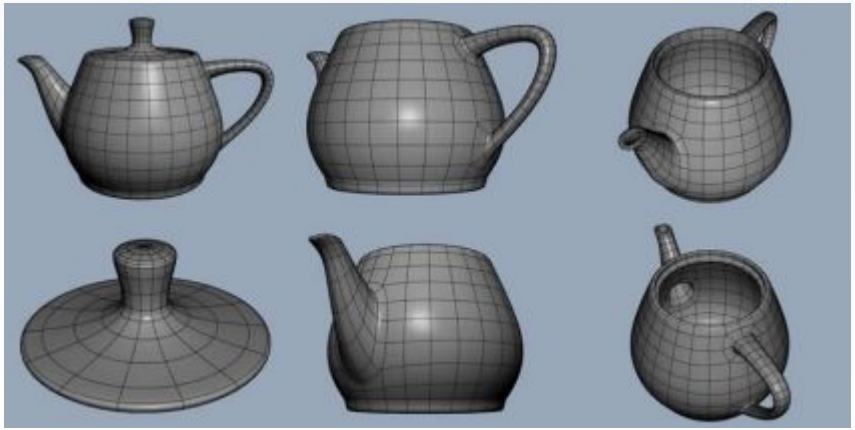
- Precisa de muitos polígonos para representar formas suaves ou detalhadas;
- Difícil edição;

Superfícies paramétricas



$$p(u,v) = [x(u,v), y(u,v), z(u,v)]$$

Exemplos: planos, cilindros e superfícies



Superfícies paramétricas

Por que são melhores que as malhas?

- Mais compactas;
- Mais fácil de controlar;
- “Mais fácil de construir”;

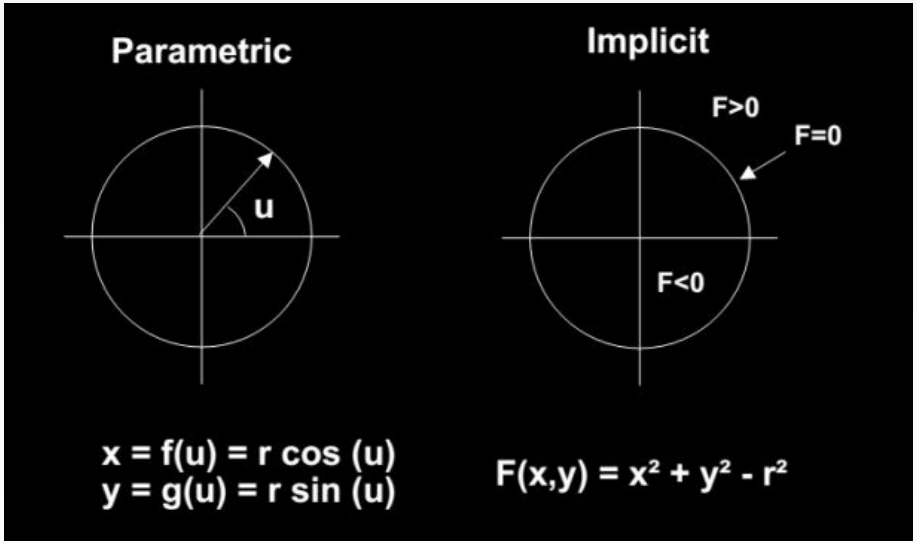
Problemas:

- Funciona bem somente em superfícies curvas e suaves;
- Demora mais para renderizar;
- Dificulta os testes de colisão;

Superfícies Implícitas

Duas formas diferentes de representar um círculo

- Fácil de calcular colisões;
- Difícil de modelar;



Questionário